UNIVERSIDAD TECNOLÓGICA DEL VALLE DE TOLUCA

Dirección de Carrera: Tecnologías de la Información y Comunicación

Programa educativo: Ingeniería en Desarrollo y Gestión de Software

Periodo: Enero - Abril, 2026

Cuatrimestre: Octavo

Estudiante:

Díaz González Jaime

Grupo: IDGS - 81

Asignatura: Seguridad en el Desarrollo de Aplicaciones

Nombre del docente: Mtro. Jorge Luis Galindo Lara

Ubicación: Santa María Atarasquillo, Lerma Estado de México

Authentication Bypass vía SQL Injection

Autor: Jaime Díaz González
Fecha: 3 de febrero de 2026

1. Resumen Ejecutivo

La vulnerabilidad conocida como "Authentication Bypass via SQL Injection" ocurre cuando una aplicación web no protege correctamente la información que recibe del usuario en un formulario de inicio de sesión. En términos simples, significa que un atacante puede ingresar texto especial en los campos de usuario o contraseña para engañar al sistema y lograr que lo deje entrar sin conocer las credenciales reales.

Esto sucede cuando el sistema construye consultas a la base de datos juntando directamente lo que escribe el usuario con el código SQL, sin aplicar mecanismos de protección adecuados. Como resultado, el atacante puede modificar la lógica de la consulta y hacer que el sistema crea que la validación fue exitosa.

En la demostración se muestra un caso típico: un formulario en la ruta /login que envía un nombre de usuario y una contraseña al servidor. Si la consulta SQL es vulnerable, es posible introducir un valor manipulado que haga que la condición de verificación siempre sea verdadera, permitiendo el acceso sin una cuenta válida.

El impacto puede ser grave, ya que una persona no autorizada podría acceder a información sensible o incluso a cuentas con privilegios administrativos. Para evitar este problema, se recomienda utilizar consultas preparadas (prepared statements), validar correctamente los datos de entrada, limitar los permisos de la base de datos y mantener monitoreo y registros adecuados.

2. Introducción y Contextualización

Definición del Problema

Una inyección SQL es un problema de seguridad que ocurre cuando una aplicación web no controla correctamente lo que escribe el usuario en un formulario. Si el sistema toma ese texto y lo coloca directamente dentro de una consulta a la base de datos, sin protección, una persona puede escribir comandos especiales para cambiar el comportamiento del sistema.

En el caso de "authentication bypass", esto significa que el atacante puede engañar al sistema de inicio de sesión. En lugar de verificar correctamente el usuario y la contraseña, la base de datos recibe una instrucción modificada que hace que la validación siempre sea verdadera, permitiendo el acceso sin tener credenciales reales.

Clasificación OWASP

Esta vulnerabilidad forma parte de los principales riesgos de seguridad identificados por OWASP (Open Worldwide Application Security Project), una organización que publica una lista llamada "OWASP Top 10" con las fallas más críticas en aplicaciones web. La inyección SQL y los problemas de control de acceso se encuentran entre las categorías más graves debido a su alto impacto y facilidad de explotación.

Impacto de Negocio

Si se explota, la empresa puede sufrir:

- Acceso no autorizado a cuentas de usuarios, incluyendo cuentas de administradores.
- Robo de información sensible, como datos personales, contraseñas o información confidencial.
- Modificación o eliminación de datos importantes dentro del sistema.
- Daño a la reputación de la organización y posibles sanciones legales o regulatorias.

3. Análisis Técnico de la Vulnerabilidad

Mecanismo de Explotación

1. El atacante identifica un endpoint de autenticación (ej. POST /login) que acepta username y password.

2. La aplicación construye y ejecuta una consulta SQL concatenando directamente esas entradas, por ejemplo:

```
SELECT id, role FROM users WHERE username = '$username' AND password = '$password';
```

3. El atacante envía un payload que altera la lógica de la consulta (por ejemplo, ' OR '1'='1' --) para hacer que la cláusula WHERE siempre sea verdadera.

4. La consulta devuelve una fila (o la primera fila), la aplicación interpreta la respuesta como autenticación exitosa y crea una sesión para el atacante.

Análisis de Código Vulnerable

Ejemplo en PHP - vulnerable:

```
// vulnerable_login.php

$username = $_POST['username'];

$password = $_POST['password'];

$sql = "SELECT id, role FROM users WHERE username = '$username' AND
password = '$password'";

$result = $db->query($sql); if

($result->num_rows > 0) {

   // login success
```

Vectores de Ataque

- Formularios de login web (POST/GET) con validación insuficiente.
- Endpoints API REST que aceptan JSON con credenciales.
- Parámetros de URL usados por servicios internos.
- Campos de entrada en aplicaciones móviles que envían a backend vulnerable.

4. Entorno de la Demostración (PoC)

Tecnologías Utilizadas

- Lenguaje: PHP
- Base de datos: MySQL

- Servidor Web: Apache

Herramientas de Pentesting

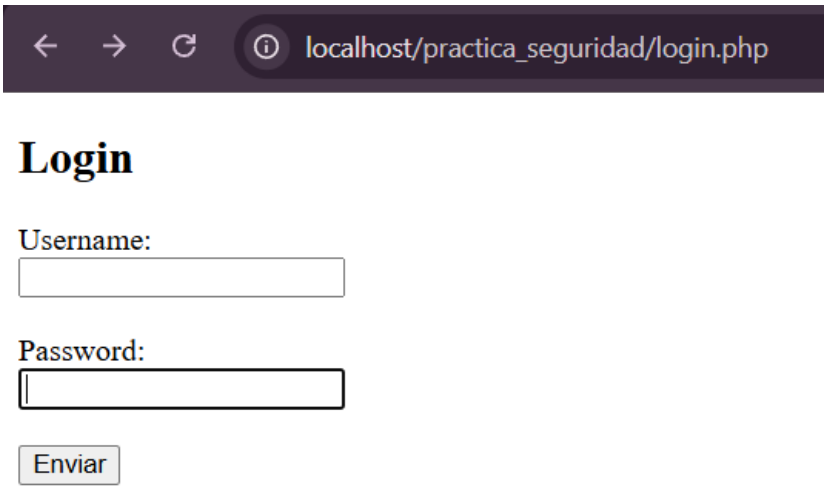
- Script de inyección consulta SQL

Arquitectura

5. Guía de la Demostración Paso a Paso (P9oC)

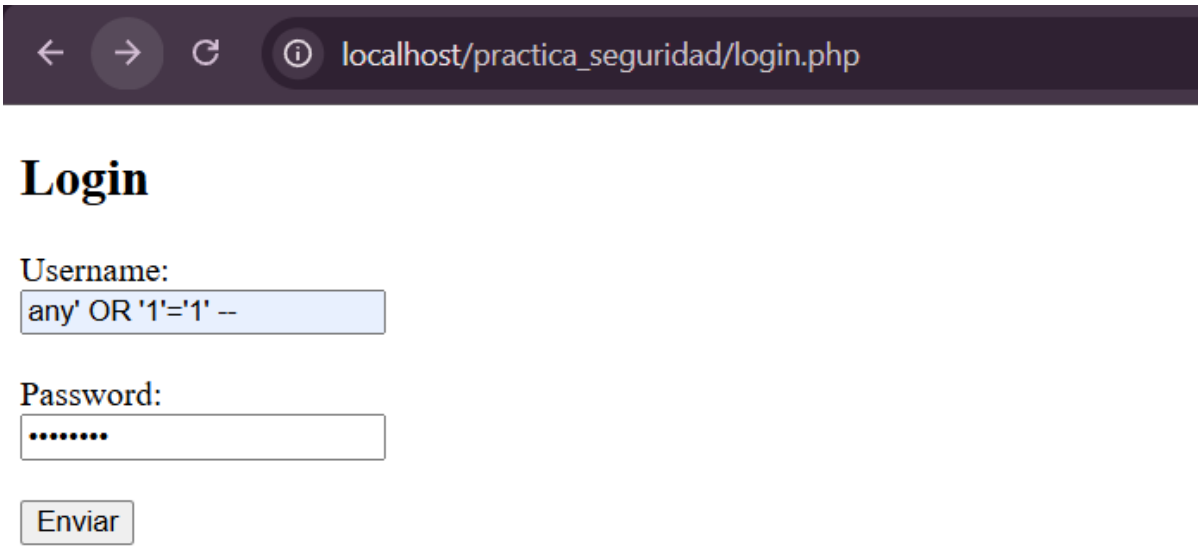
Estado Inicial

La aplicación presenta un formulario en /login con campos username y password. Internamente, el backend construye la consulta SQL concatenando variables sin binding.



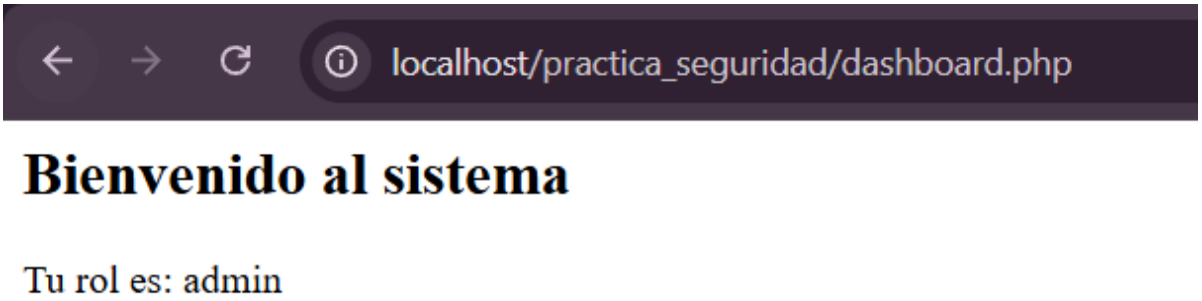
Inyección / Explotación

any' OR '1'='1' –



Resultado Esperado

La consulta SQL manipulada devuelve al menos una fila. La aplicación crea una sesión y el atacante obtiene acceso sin credenciales válidas.



6. Estrategias de Mitigación y Remediación

Solución a nivel de Código

Usar sentencias preparadas (prepared statements) con binding de parámetros.

PHP corrección:

```
$username = $_POST['username'] ?? '';

$password = $_POST['password'] ?? '';

if (strlen($username) > 20 || strlen($password) > 20) {

    die("Entrada inválida");

}

$stmt = $db->prepare("SELECT id, role FROM users WHERE username = ? AND
password = ?");

$stmt->bind_param("ss", $username, $password);

$stmt->execute();

$result = $stmt->get_result();

if ($result && $result->num_rows > 0) {

    $row = $result->fetch_assoc();

    $_SESSION['user_id'] = $row['id'];

    $_SESSION['role'] = $row['role'];

    header("Location: dashboard.php");

    exit();

} else {

    echo "Credenciales incorrectas";

}

$stmt->close();
```

Validación y Normalización de Entrada

- Validar tipos y longitudes (ej. usernames alfanuméricos, longitud máxima razonable).
- Rechazar o normalizar caracteres especiales según el contexto.

```
<label>Username:</label><br>
<input
    type="text"
    name="username"
    maxlength="30"
    pattern="^[a-zA-Z0-9_]{3,20}$"
    placeholder="Solo letras, números y guion bajo (3-30 caracteres)"
    required>
<br><br>

<label>Password:</label><br>
<input
    type="password"
    name="password"
    maxlength="20"
    required>
<br><br>
```

Gestión de Contraseñas

- No almacenar contraseñas en texto plano, usar hashing (Argon2, bcrypt, scrypt) con salt.
- Comparar hash en servidor, nunca incluir password en consultas SQL para comparación directa sin hashing.

7. Conclusiones

La inyección SQL que permite evadir autenticación sigue siendo una vulnerabilidad de alto riesgo por su simplicidad de explotación y el impacto que puede causar en la confidencialidad e integridad de los sistemas. La corrección es principalmente de ingeniería (uso de queries parametrizadas y correcto manejo de credenciales).